Initialization
Reset
Controlling Time

🤩 Initialization

After installing AI2-THOR, we can initialize a controller, which will allow us to execute actions in the environment.

```
from ai2thor.controller import Controller

controller = Controller(
    agentMode="default",
    visibilityDistance=1.5,
    scene="FloorPlan212",

    # step sizes
    gridSize=0.25,
    snapToGrid=True,
    rotateStepDegrees=90,

    # image modalities
    renderDepthImage=False,
    renderInstanceSegmentation=False,

    # camera properties
    width=300,
    height=300,
    fieldOfView=90
)
```

Controller Parameters

agentMode: str = "default"

The type of agent that will interact with the scene. For iTHOR, stick with `agentMode="default"`. RoboTHOR provides the `locobot` as an `agentMode` option, and ManipulaTHOR provides the `arm` as an `agentMode` option.

visibilityDistance: float = 1.5

Used to set the maximum goal distance in meters between the agent and an object. This affects each object's `visible` property, where, when `True`, then the agent is within `visibilityDistance` of that object and the object appears in the agent's current egocentric frame.

scene: Optional[str] = None

The name of which scene to initialize. Valid iTHOR scenes are listed here. If unspecified and the `default` agent is being used, an iTHOR scene will be chosen at random.

gridSize: float = 0.25

The amount, in meters, that the agent moves after calling a move action (e.g., `MoveAhead` and `MoveBack`).

snapToGrid: bool = True

Determines whether the agent's position is snapped to a grid point after any movement action (e.g., `MoveAhead` and `TeleportFull`). Grid points are spaced out by `gridSize`. Set to `False` to allow diagonal movement.

rotateStepDegrees: float = 90

The default amount, in degrees, that the agent rotates after calling a rotate action (i.e., `RotateLeft` or `RotateRight`).

renderDepthImage: bool = False

When `True`, a depth frame is rendered and made available as an `event.depth_frame` attribute.

> **Remark**
> We require this to be explicitly passed in because rendering depth takes longer than rendering RGB alone.

renderInstanceSegmentation: bool = False

When `True`, a instance segmentation frame is rendered and made available as an `event.instance_segmentation_frame` attribute. Instance segmentation provides segmentation by object instance, where all objects in view are distinguishable.

> **Remark**
> We require this to be explicitly passed in because rendering instance segmentation takes longer than rendering RGB alone.

width: int = 300

The number of horizontal sampled pixels for each frame. This affects every rendered image frame (e.g., `event.frame`, `event.depth_frame`, `event.cv2img`).

height: int = 300

The number of vertical sampled pixels for each frame. This affects every rendered image frame (e.g., `event.frame`, `event.depth_frame`, `event.cv2img`).

fieldOfView: int = 90

Changes the camera's optical field of view, in degrees. Valid values are in the domain $(0 : 180)$. The default field of view when `agentMode="default"` is 90 degrees.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 **허용**을 선택하여 계속합니다.
로그인

> **Warning**
> Not all agent modes have the same default `fieldOfView`. For instance, the `locobot` has a default `fieldOfView` of 60 degrees.

🌑 Reset

Any `Controller` Initialization parameter can later be changed by calling the reset method. For instance, we can call:

```
controller.reset(scene="FloorPlan319", rotateStepDegrees=30)
```

The values will default to what they were upon the most recent reset or initialization. For instance, if you initialized with `fieldOfView=45`, then called `reset` with `fieldOfView=60`, calling `reset` again, without passing in `fieldOfView` will keep `fieldOfView=60`.

⏱️ Controlling Time

By default, AI2-THOR executes high-level actions that provide a response from the environment after the action completes execution. For instance, if the agent successfully executes an `OpenObject` action, the response is the state of the environment after the action has completely finished executing.

However, we also provide the ability to observe intermediate results, which is discussed in this section.

🐌 Pause Physics Auto-Simulation

`PausePhysicsAutoSim` toggles off auto-simulation in the environment, which must be called before one can control how much time passes in-between steps.

```
controller.step("PausePhysicsAutoSim")
```

👉 Advance Physics Step

`AdvancePhysicsStep` responds with an Event after simulating the environment for `timeStep` seconds beyond the current state. Note that `PausePhysicsAutoSim` must be called first, before calling `AdvancePhysicsStep`.

```
controller.step(
    action="AdvancePhysicsStep",
    timeStep=0.01
)
```

Advance Physics Step Parameters

timeStep: float = 0.01

The amount of time in seconds to advance the physics simulation. Valid values are in $(0 : 0.05]$.

🥶 Unpause Physics Auto-Simulation

`UnpausePhysicsAutoSim` toggles back on the default physics auto-simulation, which returns future events after the action finishes execution.

> **Warning**
> Unpausing auto-simulation while objects are still in motion (i.e., `inMotion=True` on an object's metadata) will freeze each object at its current location. Thus, the remaining motion of such an object will be lost.
>
> To capture the entire movement of an object, do not unpause physics with this action until all objects return `inMotion=False` in their object metadata, which can also be checked with the shorthand of `event.metadata["isSceneAtRest"]`.

```
controller.step("UnpausePhysicsAutoSim")
```